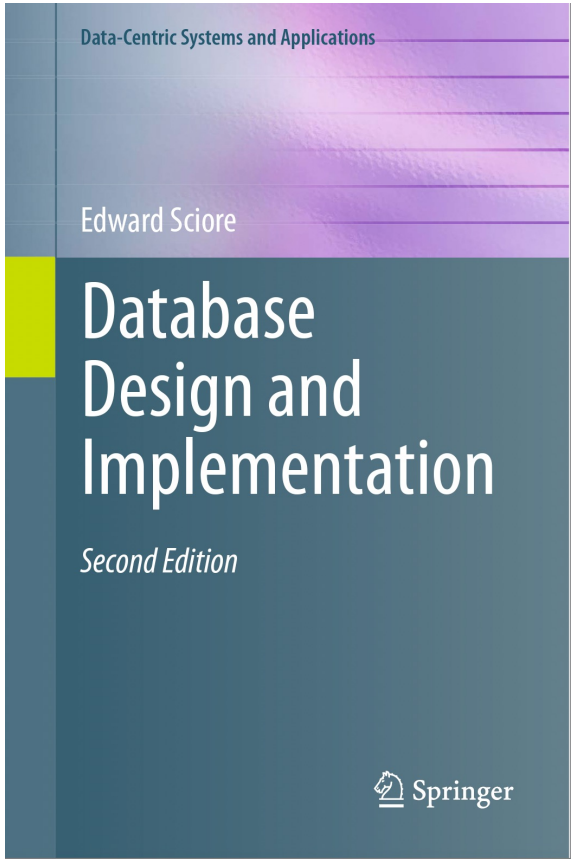
SimpleDB, parte 2

Luigi Bellomarini
luigi.bellomarini@uniroma3.it



SimpleDB, componenti

JDBC: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

- Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

- Metadata: gestisce gli schemi delle tabelle

Record: gestisce i blocchi per i record delle tabelle

Transaction: gestisce la concorrenza e l'affidabilità

Buffer: mantiene in memoria le pagine per limitare gli accessi

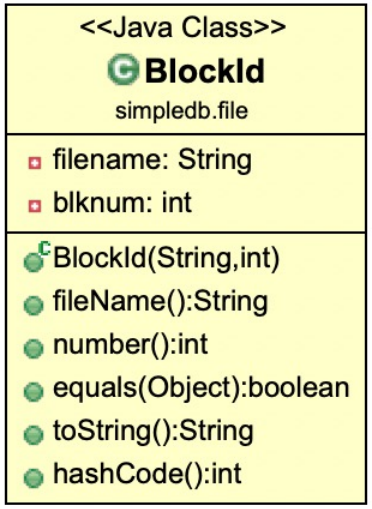
Log: tiene traccia delle operazioni, per l'affidabilità

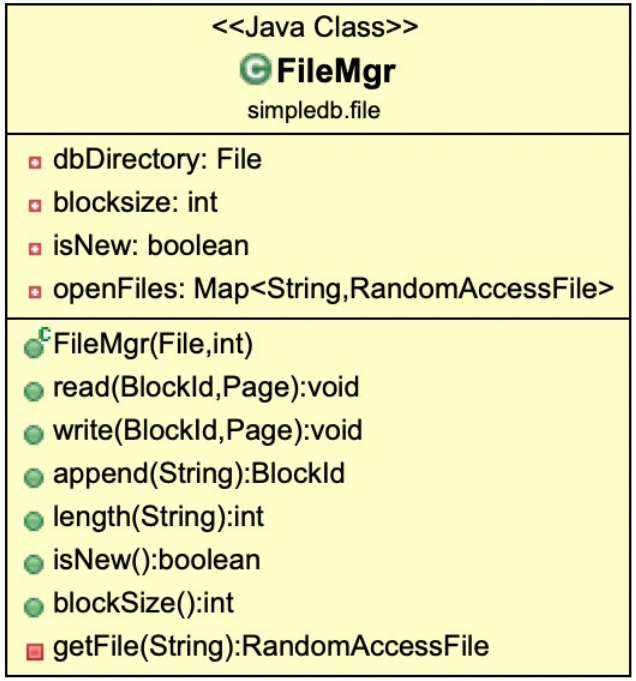
File: legge e scrive le pagine su disco

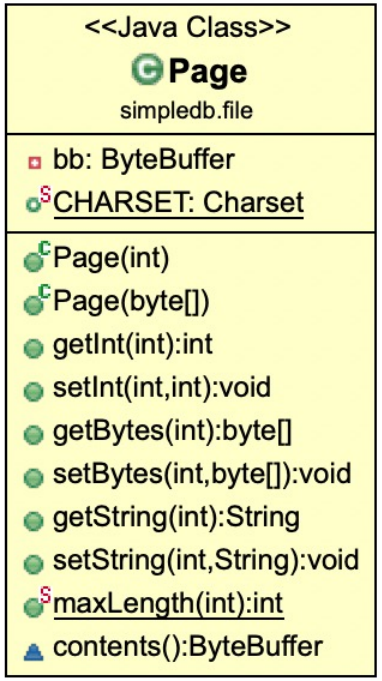
File Manager, API

```
FileMgr
    dbDirectory : File
    blocksize : int
    isNew : boolean
    openFiles : Map<String, RandomAccessFile>
    FileMgr(File, int)
    read(BlockId, Page) : void
    write(BlockId, Page) : void
    append(String) : BlockId
    length(String) : int
    isNew() : boolean
    blockSize() : int
    getFile(String) : RandomAccessFile
```

Package file







Esercizio pratico 1.1: implementare statistiche di accesso a livello di blocco (1/2)

- **Creare una classe per memorizzare le statistiche di lettura/scrittura di blocchi, ad es: simpledb.file.BlockStats**
  - La classe dovrà memorizzare il numero di blocchi letti e scritti suddividendoli per file

- **Metodi di BlockStats:**
  - toString() – che stampa la statistiche
  - logReadBlock(Block) – che aggiorna le statistiche sulla base della lettura di un blocco
  - logWrittenBlock(Block) – che aggiorna le statistiche sulla base della scrittura di un blocco
  - reset() – che azzera le statistiche memorizzate fino a quel momento

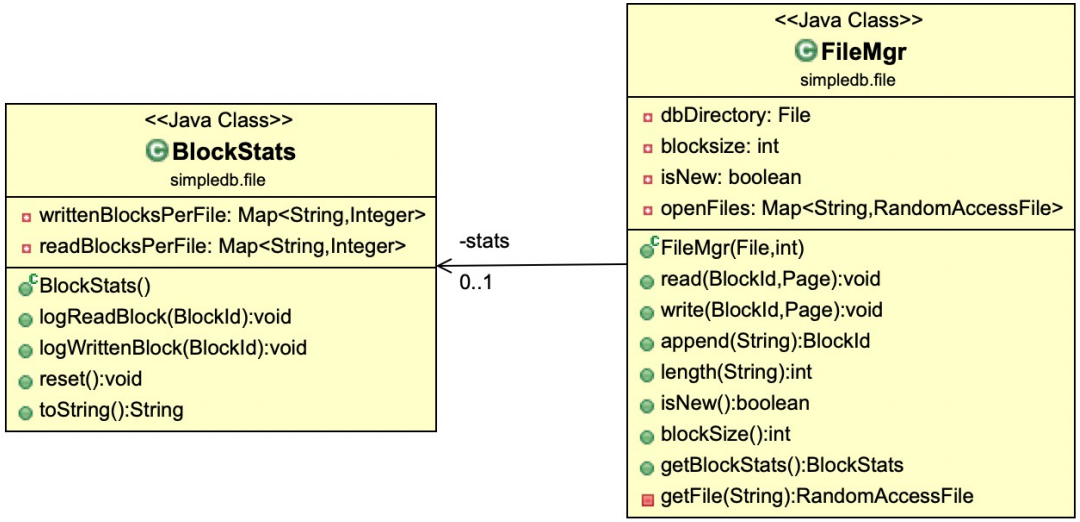
Esercizio pratico: implementare statistiche di accesso a livello di blocco (2/2)

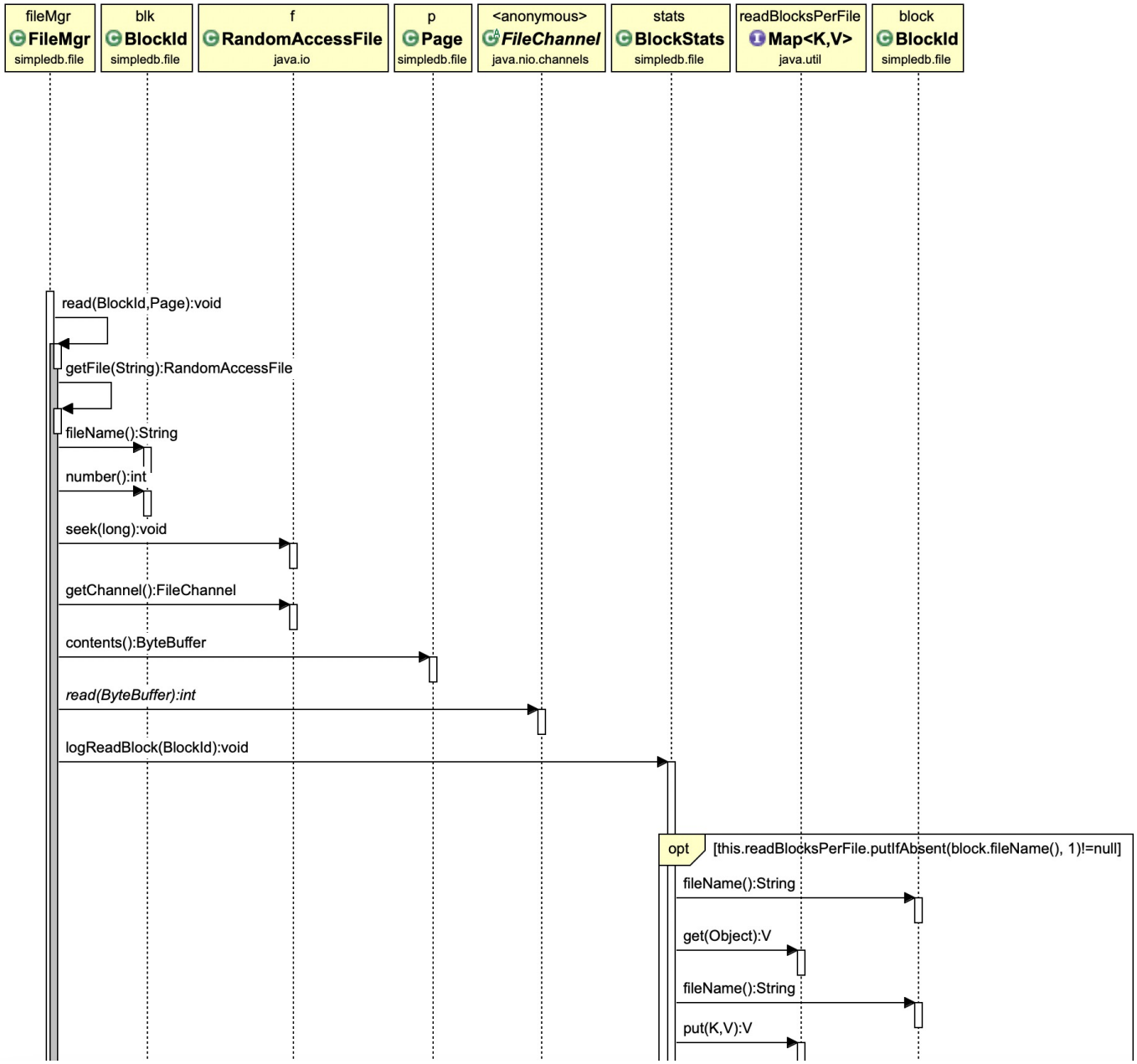
- **Modificare la classe FileMgr, in modo tale che implementi metodi per accedere alle statistiche sui blocchi.**

- **Metodi di FileMgr:**
  - getBlockStats() – che ritorna un'istanza di BlockStats
  - resetBlockStats() – che azzera le statistiche memorizzate fino a quel momento

- Nel package JDBC, modificare il metodo commit() di EmbeddedConnection per stampare le statistiche sui blocchi al commit di ogni transazione.

BlockStats


<<Java Class>>
BlockStats
simpledb.file
writtenBlocksPerFile: Map<String,Integer>
readBlocksPerFile: Map<String,Integer>
BlockStats()
logReadBlock(BlockId):void
logWrittenBlock(BlockId):void
reset():void
toString():String
-stats
0..1
<<Java Class>>
FileMgr
simpledb.file
dbDirectory: File
blocksize: int
isNew: boolean
openFiles: Map<String,RandomAccessFile>
FileMgr(File,int)
read(BlockId,Page):void
write(BlockId,Page):void
append(String):BlockId
length(String):int
isNew():boolean
blockSize():int
getBlockStats():BlockStats
getFile(String):RandomAccessFile


fileMgr
FileMgr
simpledb.file
blk
BlockId
simpledb.file
f
RandomAccessFile
java.io
p
Page
simpledb.file
<anonymous>
FileChannel
java.nio.channels
stats
BlockStats
simpledb.file
readBlocksPerFile
Map<K,V>
java.util
block
BlockId
simpledb.file
read(BlockId,Page):void
getFile(String):RandomAccessFile
fileName():String
number():int
seek(long):void
getChannel():FileChannel
contents():ByteBuffer
read(ByteBuffer):int
logReadBlock(BlockId):void
opt
[this.readBlocksPerFile.putIfAbsent(block.fileName(), 1)!=null]
fileName():String
get(Object):V
fileName():String
put(K,V):V

Memory management

- Due principi
  - ***Minimizzare gli accessi al disco***
    - Molto oneroso
    - L'utilizzo di meccanismi di caching è una tipica soluzione
    - CPU cache, disk cache
  - ***Non affidarsi alla memoria virtuale***
    - *Non permette di effettuare il recovery da crash*
    - *Può scrivere delle pagine (swap) anche se il log non è stato ancora scritto*
    - *Il SO decide quali pagine scrivere/leggere sulla base di euristiche proprie, subottime.*
    - *Il db ha conoscenza più approfondita e può realmente prendere delle decisioni.*

SimpleDB, componenti "read-only"

Remote: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

Metadata: gestisce gli schemi delle tabelle

Record: gestisce i blocchi per i record delle tabelle

Transaction: gestisce la concorrenza e l'affidabilità

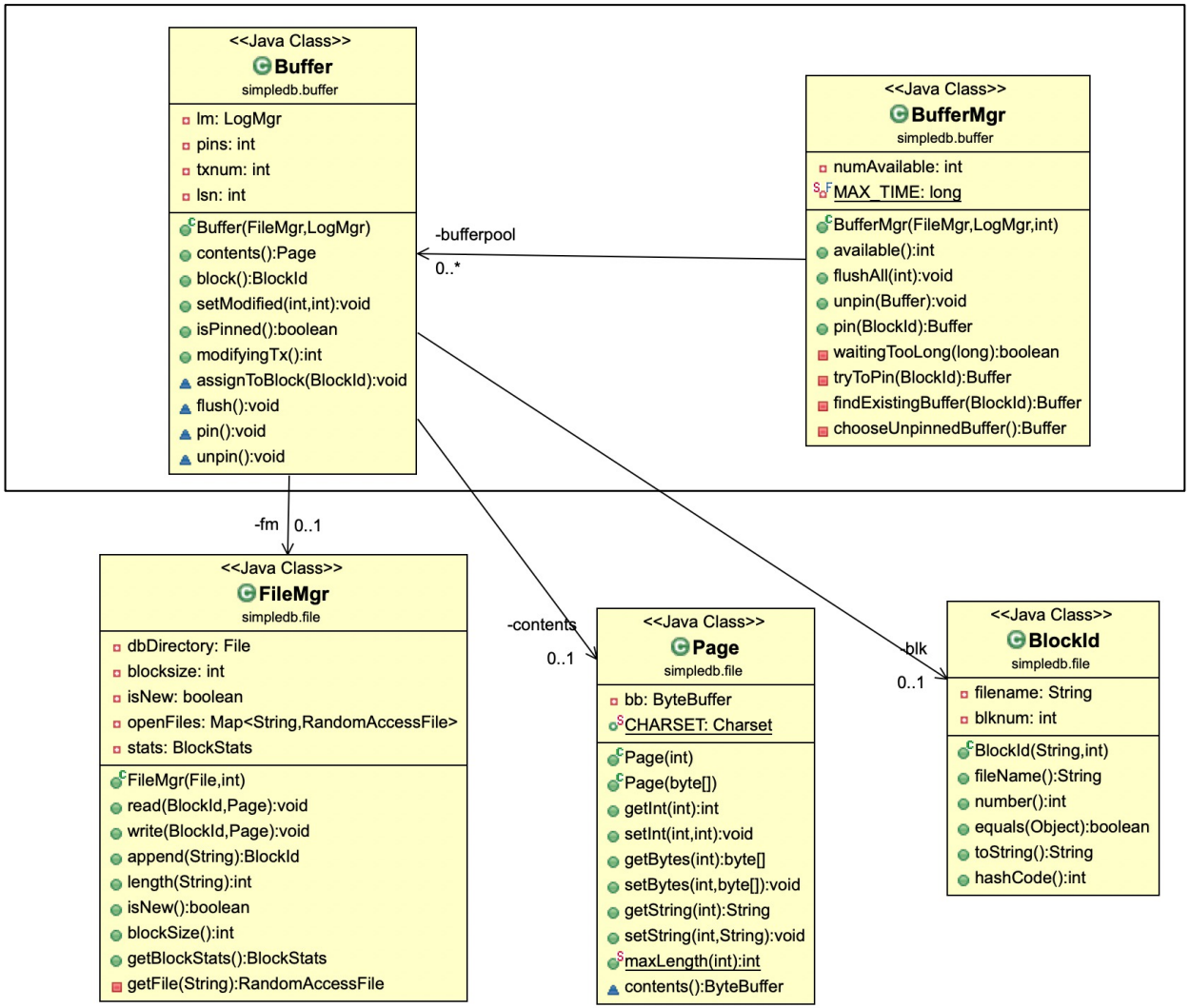
Buffer: mantiene in memoria le pagine per limitare gli accessi

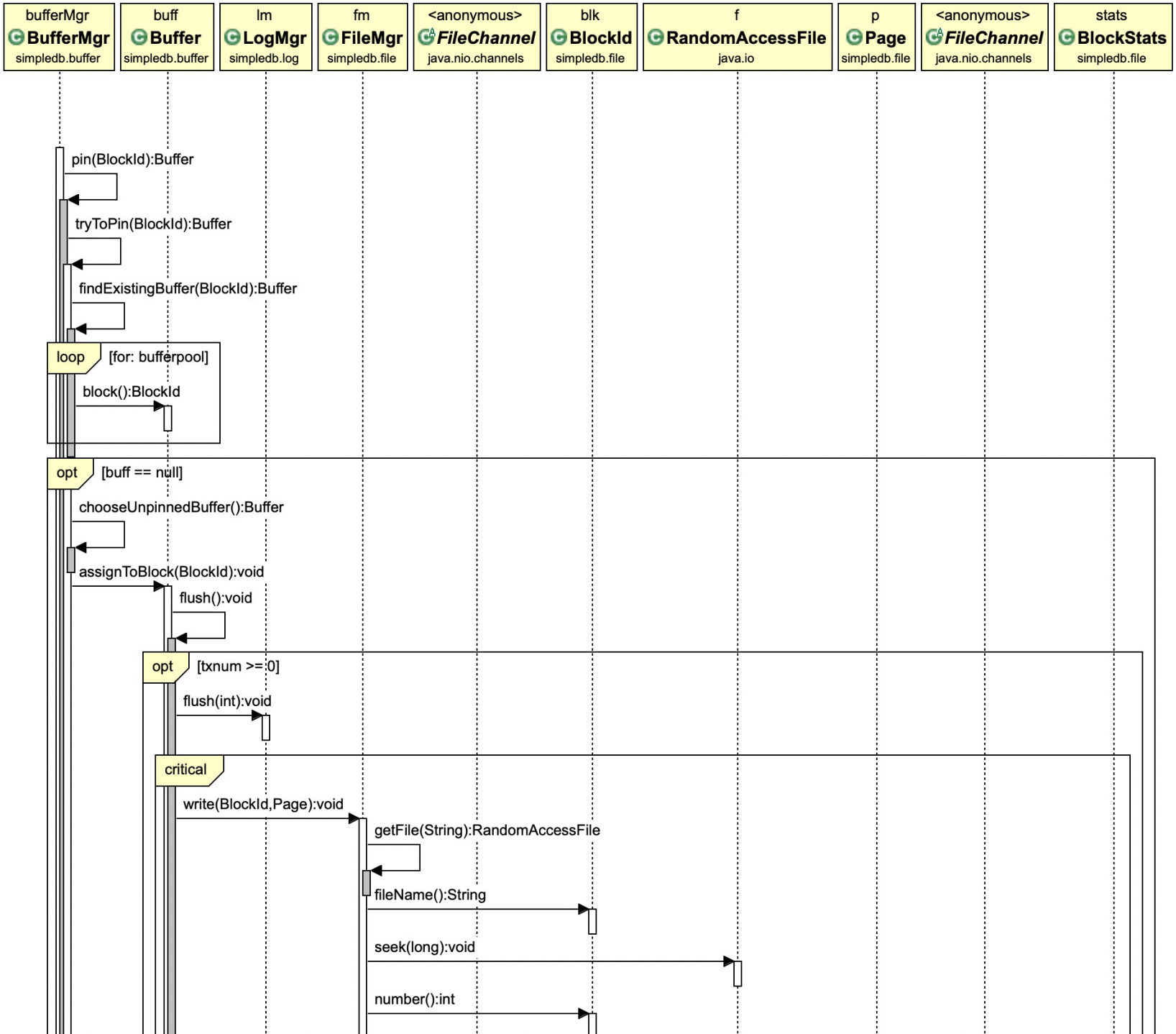
Log: tiene traccia delle operazioni, per l'affidabilità

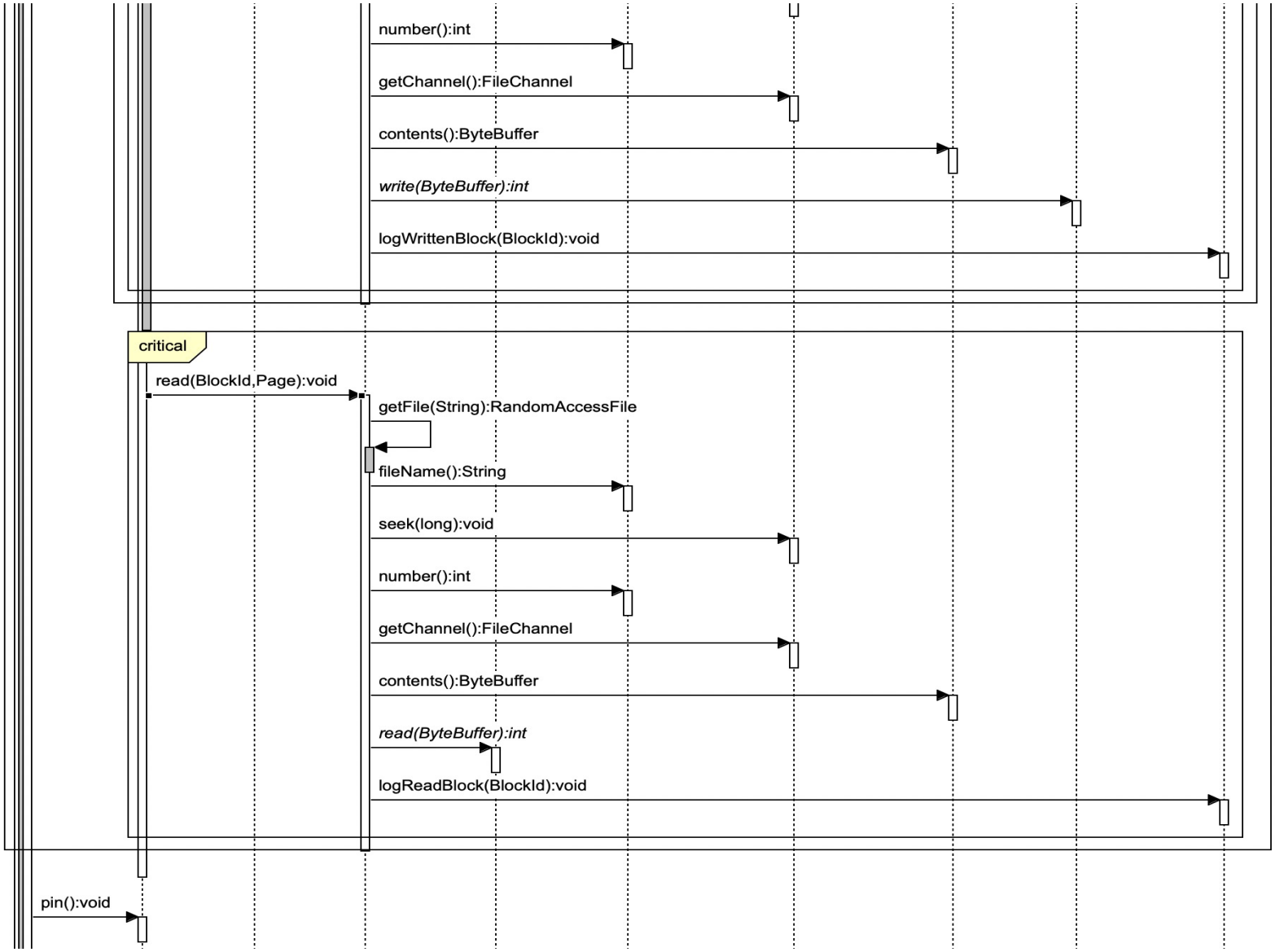
File: legge e scrive le pagine su disco

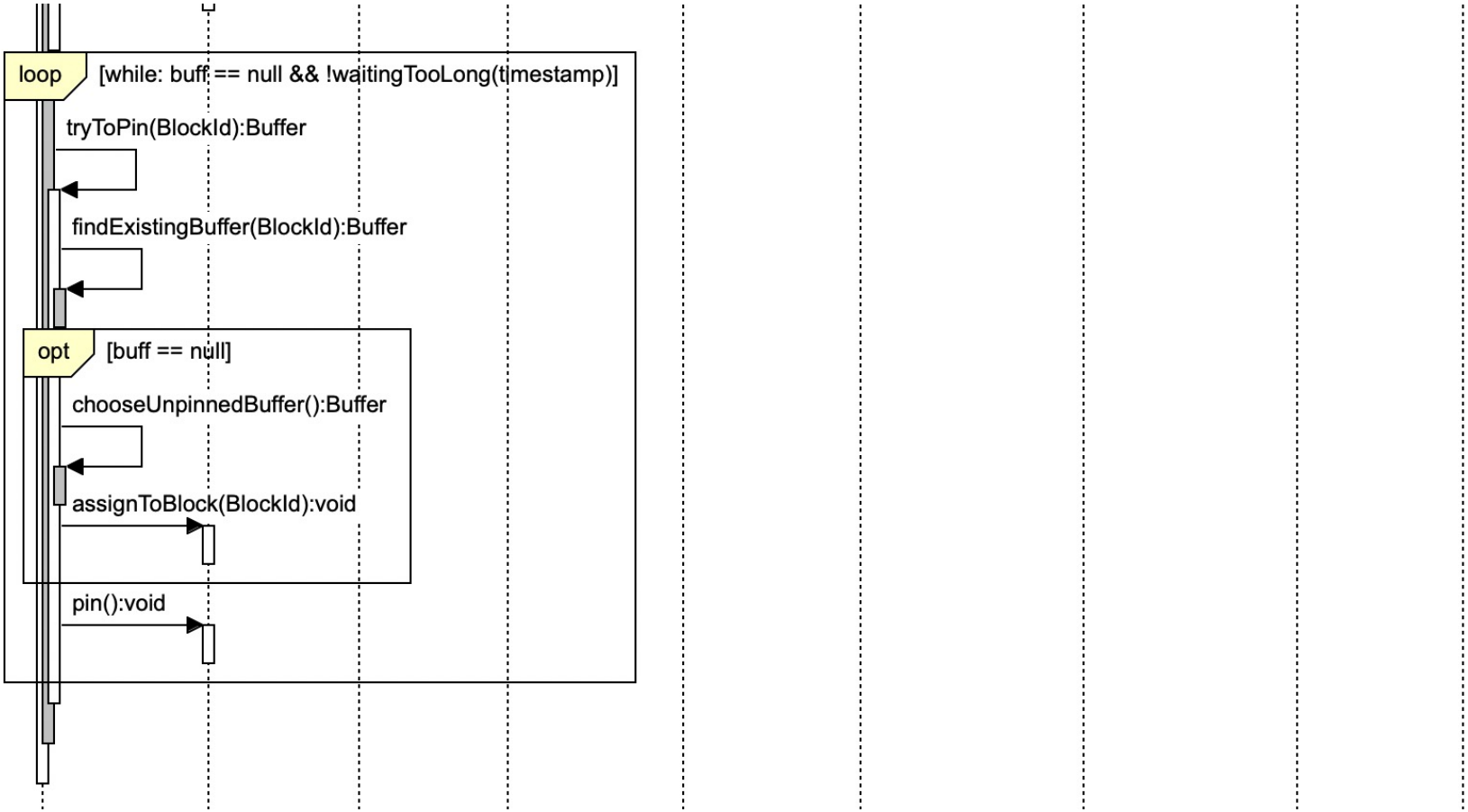
Package Buffer

buffer




bufferMgr
BufferMgr
simpledb.buffer
buff
Buffer
simpledb.buffer
lm
LogMgr
simpledb.log
fm
FileMgr
simpledb.file
<anonymous>
FileChannel
java.nio.channels
blk
BlockId
simpledb.file
f
RandomAccessFile
java.io
p
Page
simpledb.file
<anonymous>
FileChannel
java.nio.channels
stats
BlockStats
simpledb.file
pin(BlockId):Buffer
tryToPin(BlockId):Buffer
findExistingBuffer(BlockId):Buffer
loop
[for: bufferpool]
block():BlockId
opt
[buff == null]
chooseUnpinnedBuffer():Buffer
assignToBlock(BlockId):void
flush():void
opt
[txnum >= 0]
flush(int):void
critical
write(BlockId,Page):void
getFile(String):RandomAccessFile
fileName():String
seek(long):void
number():int


number():int
getChannel():FileChannel
contents():ByteBuffer
write(ByteBuffer):int
logWrittenBlock(BlockId):void
critical
read(BlockId,Page):void
getFile(String):RandomAccessFile
fileName():String
seek(long):void
number():int
getChannel():FileChannel
contents():ByteBuffer
read(ByteBuffer):int
logReadBlock(BlockId):void
pin():void


loop
[while: buff == null && !waitingTooLong(timestamp)]
tryToPin(BlockId):Buffer
findExistingBuffer(BlockId):Buffer
opt
[buff == null]
chooseUnpinnedBuffer():Buffer
assignToBlock(BlockId):void
pin():void

Buffer Manager

- Mantiene in memoria le pagine per limitare gli accessi
- **The buffer manager is the portion of the database system responsible for the pages that hold user data**
- The buffer manager allocates a fixed set of pages, called the *buffer pool*
- In order to access a block, a client interacts with the buffer manager
  - The client asks the buffer manager to *pin* a page from the buffer pool to that block
  - The client accesses the contents of the page as much as it desires
  - When the client is done with the page, it tells the buffer manager to *unpin* it
- A page is *pinned* if some client is currently pinning it; otherwise the page is *unpinned*
- *A pinned buffer page is in use by its clients*
- *An unpinned buffer page is available for reuse*

Buffer Manager

- When a client asks the buffer manager **to pin a page to a block**, the buffer manager will encounter one of these four possibilities:
  - The contents of the block is in some page in the buffer, and:
    - the page is pinned, or
    - the page is unpinned
  - The contents of the block is not currently in any buffer, and:
    - there exists at least one unpinned page in the buffer pool, or
    - all pages in the buffer pool are pinned

- The first case occurs when one or more clients are currently accessing the contents of the block. Since a page can be pinned by multiple clients, the buffer manager simply adds another pin to the page, and returns the page to the client. Each client that is pinning the page is free to concurrently read and modify its values.

Buffer Manager

- The contents of the block is in some page in the buffer, and the page is unpinned:
  occurs when the client(s) that were using the buffer have now finished with it, but the buffer has not yet been reused. Since the contents of the block are still in the buffer page, the buffer manager can reuse the page by simply pinning it and returning it to the client

- The contents of the block is not currently in any buffer, and there exists at least one unpinned page in the buffer pool:
  requires the buffer manager to read the block from disk into a buffer page. Several steps are involved. The buffer manager must first select a page to reuse; this page must be unpinned (because the pinned pages are still being used by clients). Second, if the selected page has been modified, then the buffer manager must first write the page contents back to disk; this action is called *flushing* the page. Finally, the block can be read into the selected page, and the page can be pinned.

Buffer Manager

When a client asks the buffer manager to pin a page to a block

- Fourth case - The contents of the block is not currently in any buffer, and all pages in the buffer pool are pinned:
  occurs when the buffers are heavily used, such as in the query-processing. In this case, the buffer manager cannot satisfy the client request. The best solution is for the buffer manager to place the client on a wait list.
  When a buffer becomes unpinned, the buffer manager can pin that page to the block, and the client can continue.

Buffer Manager

- Each page in the buffer pool has associated status information, such as whether it is pinned, and if so, what block it is pinned to. A *buffer* is the object that contains this information.
- Every page in the buffer pool has its own buffer.
- Each buffer observes the changes to its page, and is responsible for writing its modified page to disk.
- if the page is modified several times, then it is more efficient to write the page once, after all modifications have been made. A reasonable strategy therefore is to have the buffer postpone writing its page to disk until the page is unpinned
- There are only two reasons why a buffer will ever need to write a modified page to disk: either the page is being replaced because the buffer is getting pinned to a different block (as in the third case); or the recovery manager needs to write its contents to disk in order to guard against a possible system crash

Buffer replacement strategy

- The pages in the buffer pool begin unallocated. As pin requests arrive, the buffer manager primes the buffer pool by assigning the requested blocks to unallocated pages.
- Once all pages have been allocated, the buffer manager will have to begin replacing pages. The buffer manager can choose to replace any page in the buffer pool, provided that it is unpinned
- Each client has the responsibility to “be a good citizen” and unpin a buffer as soon as it is no longer needed
- If more than one buffer page is unpinned, then the buffer manager must decide which one to replace. This choice affects the number of disk accesses that will be performed

Strategia di rimpiazzo (Buffer replacement strategy)

- SimpleDB usa
  - "no steal"
  - strategia naïve

Strategia di rimpiazzo (demo su SimpleDB)

- Analizziamo la classe BufferTest

Buffer replacement strategy

- Since the buffer manager cannot predict which pages will be accessed next, it is forced to guess.
- But the buffer manager *knows* whether a page is currently being used or not, because the unused pages are exactly the unpinned pages
- The buffer replacement strategy only has to choose from among the currently-unwanted pages, which is far less critical
- Four general replacement strategies:
  - naïve (cerca la prima pagina libera)
  - FIFO (utilizza la pagina che è stata caricata da più tempo)
  - LRU (utilizza la pagina che è stata usata meno di recente)
  - clock (fa una scansione, come nel caso naïve, ma non dall'inizio, bensì dalla pagina successiva a quella del rimpiazzo precedente)

Buffer replacement strategy

pin(10); pin(20); pin(30); pin(40); unpin(20);
pin(50); unpin(40); unpin(10); unpin(30); unpin(50);

buffer	0	1	2	3
block	10	50	30	40
Time read in	1	6	3	4
Time unpinned	8	10	9	7

Naïve strategy

pin(60), unpin(60), pin(70), unpin(70), pin(60), unpin(60),…

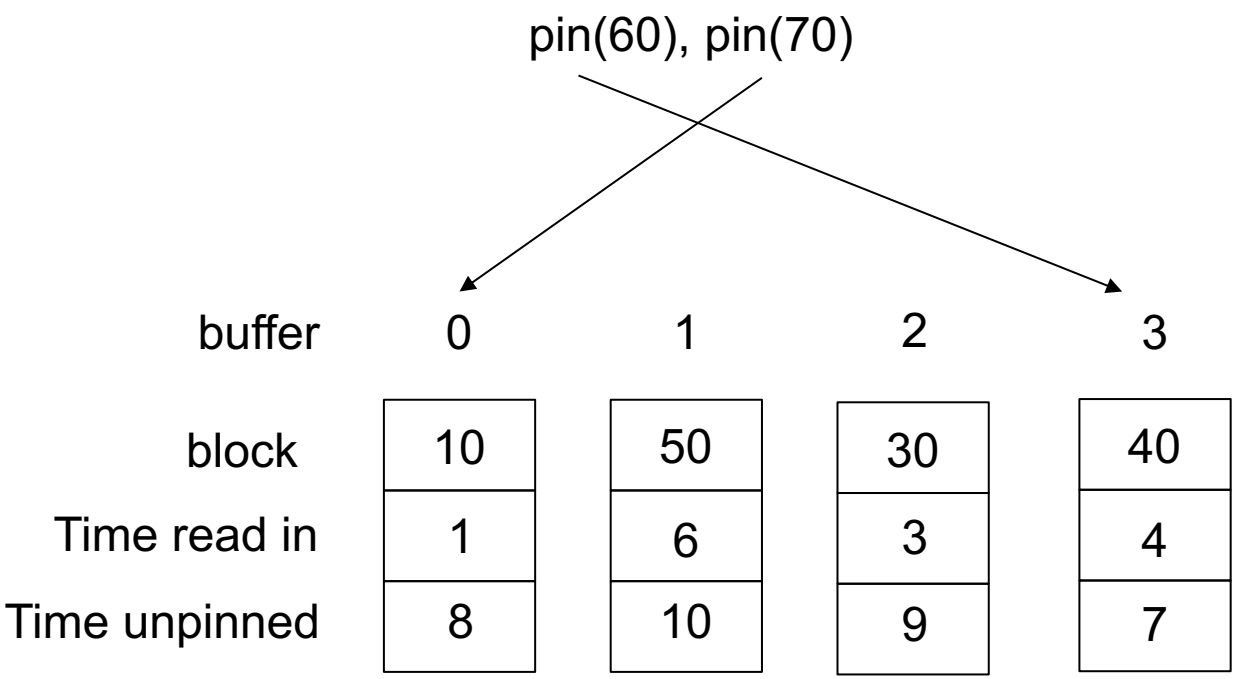
buffer	0	1	2	3
block	10	50	30	40
Time read in	1	6	3	4
Time unpinned	8	10	9	7

FIFO strategy

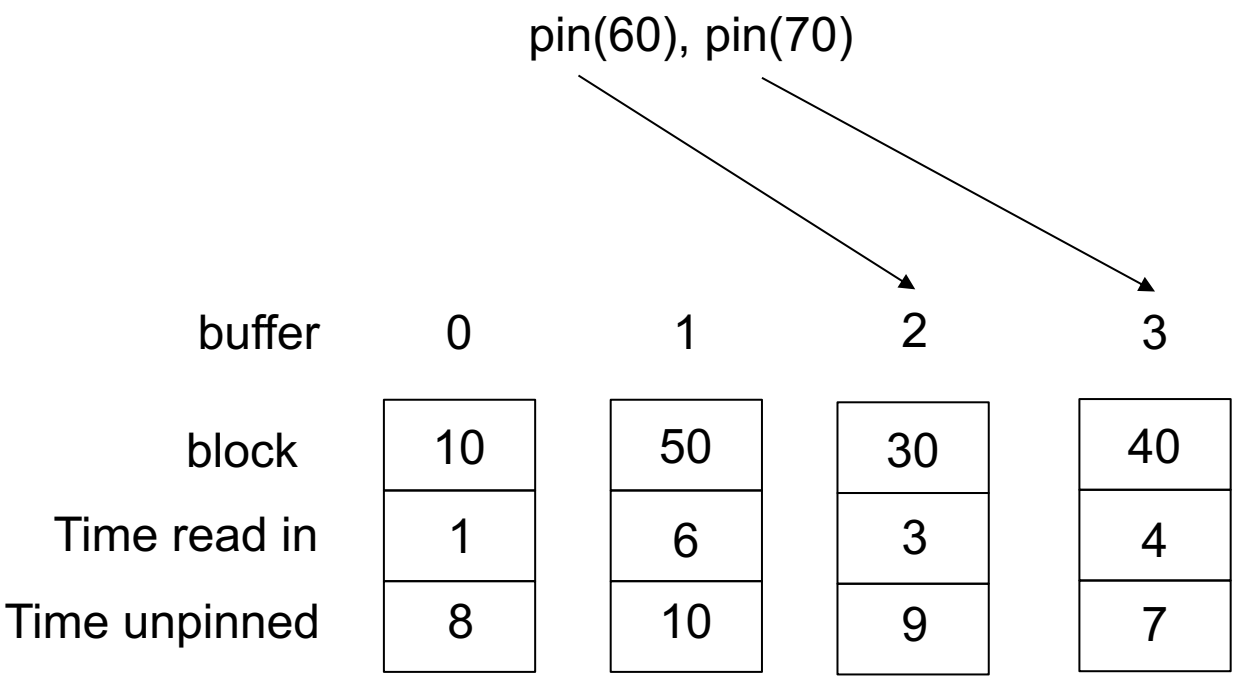
pin(60), pin(70)

buffer	0	1	2	3
block	10	50	30	40
Time read in	1	6	3	4
Time unpinned	8	10	9	7

LRU strategy



Clock strategy



Esercizio pratico 2.1

- SimpleDB utilizza la strategia naïve.
- Implementare **le altre strategie di rimpiazzo** (FIFO, LRU, clock) e renderle attivabili opzionalmente in SimpleDB.
- Considerare l'esercizio d'esame Prova parziale del 28/03/2012
- Implementare una classe di test che:
  - Imposta lo stato del buffer pool descritto nella prova parziale
  - Esegue tutte le operazioni di modifica del buffer descritte attivando le varie strategie
  - Produce in output la tabella di informazioni richiesta dall'esercizio.